API

API Documentation

Overview

The Universal Multi-Format Multi-Language Ebook Translation System provides a comprehensive REST API for translation services, distributed processing, and system management.

Base URL

```
Production: https://api.translator.digital
Development: http://localhost:8080
```

Authentication

The API uses JWT (JSON Web Token) authentication. Include the token in the Authorization header:

```
Authorization: Bearer <your-jwt-token>
```

Getting a Token

```bash
curl -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "username": "admin",
    "password": "password"
  }'
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires_in": 3600,
  "user": {
    "id": 1,
    "username": "admin",
```

```
    "role": "admin"
  }
}
```

API Endpoints

Authentication

POST /api/v1/auth/login

Authenticate user and return JWT token.

Request Body:

```
{
  "username": "string",
  "password": "string"
}
```

Response:

```
{
  "token": "string",
  "expires_in": 3600,
  "user": {
    "id": 1,
    "username": "string",
    "role": "string"
  }
}
```

POST /api/v1/auth/refresh

Refresh JWT token.

Headers: `Authorization: Bearer <token>`

Response:

```
{
  "token": "string",
  "expires_in": 3600
}
```

Translation Services

POST /api/v1/translate/translate

Translate text from source language to target language.

Request Body:

```
{
  "text": "Hello, world!",
  "source_lang": "en",
  "target_lang": "sr",
  "provider": "openai",
  "options": {
    "model": "gpt-4",
    "temperature": 0.7,
    "max_tokens": 1000
  }
}
```

Response:

```
{
  "translated_text": "Здраво, свете!",
  "source_lang": "en",
  "target_lang": "sr",
  "provider": "openai",
  "confidence": 0.95,
  "processing_time_ms": 1200,
  "cost": 0.002
}
```

POST /api/v1/translate/batch

Batch translate multiple texts.

Request Body:

```
{
  "texts": [
    "Hello, world!",
    "How are you?",
    "Good morning"
  ],
  "source_lang": "en",
  "target_lang": "sr",
  "provider": "openai",
  "options": {
    "model": "gpt-4"
  }
}
```

Response:

```
{
  "results": [
    {
      "translated_text": "Здраво, свете!",
      "confidence": 0.95
```

```
    },
    {
      "translated_text": "Како си?",
      "confidence": 0.92
    },
    {
      "translated_text": "Добро јутро",
      "confidence": 0.94
    }
  ],
  "total_processed": 3,
  "processing_time_ms": 3500
}
```

Ebook Processing

POST /api/v1/ebook/upload

Upload ebook file for processing.

Request: `multipart/form-data`

- `file`: The ebook file (FB2, EPUB, TXT, HTML)
- `source_lang`: Source language code
- `target_lang`: Target language code
- `provider`: Translation provider
- `options`: Additional options (JSON string)

Response:

```
{
  "upload_id": "uuid-string",
  "filename": "example.fb2",
  "size": 1024000,
  "format": "fb2",
  "status": "uploaded",
  "created_at": "2024-01-15T10:30:00Z"
}
```

GET /api/v1/ebook/{upload_id}/status

Get processing status for uploaded ebook.

Response:

```
{
  "upload_id": "uuid-string",
  "status": "processing",
  "progress": 75,
  "current_stage": "translating",
  "estimated_completion": "2024-01-15T10:35:00Z",
  "pages_processed": 150,
```

```
  "total_pages": 200
}
```

GET /api/v1/ebook/{upload_id}/download

Download processed ebook.

Query Parameters:

- `format`: Output format (fb2, epub, pdf, docx)
- `script`: Script for Serbian (cyrillic, latin)

Response: Binary file download

Translation Providers

GET /api/v1/providers

List available translation providers.

Response:

```
{
  "providers": [
    {
      "name": "openai",
      "display_name": "OpenAI GPT",
      "models": ["gpt-3.5-turbo", "gpt-4", "gpt-4-turbo"],
      "features": ["translation", "context-aware", "high-quality"]
    },
    {
      "name": "anthropic",
      "display_name": "Anthropic Claude",
      "models": ["claude-3-sonnet", "claude-3-opus"],
      "features": ["translation", "literary-style", "context-
        aware"]
    }
  ]
}
```

GET /api/v1/providers/{provider}/models

Get available models for a provider.

Response:

```
{
  "provider": "openai",
  "models": [
    {
      "name": "gpt-4",
      "display_name": "GPT-4",
```

```
      "max_tokens": 8192,
      "cost_per_1k_tokens": 0.03,
      "supported_languages": ["en", "ru", "sr", "zh", "es", "fr"]
    },
    {
      "name": "gpt-3.5-turbo",
      "display_name": "GPT-3.5 Turbo",
      "max_tokens": 4096,
      "cost_per_1k_tokens": 0.002,
      "supported_languages": ["en", "ru", "sr", "zh", "es", "fr"]
    }
  ]
}
```


Language Support

GET /api/v1/languages

Get supported languages.

Response:


```
{
  "languages": [
    {
      "code": "en",
      "name": "English",
      "native_name": "English"
    },
    {
      "code": "ru",
      "name": "Russian",
      "native_name": "Русский"
    },
    {
      "code": "sr",
      "name": "Serbian",
      "native_name": "Српски"
    }
  ]
}
```


GET /api/v1/languages/{source}/targets

Get available target languages for a source language.

Response:


```
{
  "source_language": {
    "code": "ru",
    "name": "Russian"
```

```
  },
  "target_languages": [
    {
      "code": "sr",
      "name": "Serbian",
      "quality_score": 0.95
    },
    {
      "code": "en",
      "name": "English",
      "quality_score": 0.98
    }
  ]
}
```

Quality and Verification

POST /api/v1/verification/check

Check translation quality.

Request Body:

```
{
  "original_text": "Hello, world!",
  "translated_text": "Здраво, свете!",
  "source_lang": "en",
  "target_lang": "sr",
  "provider": "openai"
}
```

Response:

```
{
  "quality_score": 0.92,
  "confidence": 0.95,
  "issues": [],
  "suggestions": [
    {
      "type": "style",
      "message": "Translation style is appropriate for the
        context",
      "severity": "info"
    }
  ],
  "verification_passed": true
}
```

Distributed Processing

GET /api/v1/distributed/status

Get distributed system status.

Response:

```
{
  "cluster_status": "healthy",
  "total_nodes": 3,
  "active_nodes": 3,
  "total_translations": 1500,
  "processing_rate": 45.2,
  "nodes": [
    {
      "id": "node-1",
      "status": "active",
      "load": 0.75,
      "translations_processed": 500,
      "last_seen": "2024-01-15T10:30:00Z"
    }
  ]
}
```

POST /api/v1/distributed/translate

Submit translation request to distributed system.

Request Body:

```
{
  "text": "Large text for distributed processing...",
  "source_lang": "en",
  "target_lang": "sr",
  "priority": "normal",
  "callback_url": "https://your-app.com/callback"
}
```

Response:

```
{
  "request_id": "uuid-string",
  "status": "queued",
  "estimated_completion": "2024-01-15T10:32:00Z",
  "queue_position": 5
}
```

Monitoring and Metrics

GET /api/v1/metrics/health

System health check.

Response:

```
{
  "status": "healthy",
  "version": "1.0.0",
  "uptime": 86400,
  "services": {
    "database": "healthy",
    "cache": "healthy",
    "translation_providers": "healthy",
    "distributed_system": "healthy"
  }
}
```

GET /api/v1/metrics/stats

Get system statistics.

Response:

```
{
  "total_translations": 10000,
  "translations_today": 500,
  "active_users": 25,
  "success_rate": 0.98,
  "average_processing_time_ms": 1200,
  "most_used_providers": [
    {"name": "openai", "usage_count": 5000},
    {"name": "anthropic", "usage_count": 3000}
  ],
  "most_used_language_pairs": [
    {"pair": "ru-sr", "usage_count": 4000},
    {"pair": "en-sr", "usage_count": 3000}
  ]
}
```

Error Handling

Error Response Format

All error responses follow this format:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
```

```
    "message": "Invalid input parameters",
    "details": [
      {
        "field": "source_lang",
        "message": "Invalid language code"
      }
    ],
    "request_id": "uuid-string"
  }
}
```

Common Error Codes

Code	HTTP Status	Description
`VALIDATION_ERROR`	400	Invalid input parameters
`UNAUTHORIZED`	401	Authentication required or failed
`FORBIDDEN`	403	Insufficient permissions
`NOT_FOUND`	404	Resource not found
`RATE_LIMITED`	429	Too many requests
`PROVIDER_ERROR`	502	Translation provider error
`INTERNAL_ERROR`	500	Internal server error
`SERVICE_UNAVAILABLE`	503	Service temporarily unavailable

Rate Limiting

API requests are rate-limited to ensure fair usage:

- **Free Tier**: 100 requests per hour
- **Basic Tier**: 1,000 requests per hour
- **Pro Tier**: 10,000 requests per hour
- **Enterprise**: Unlimited

Rate limit headers are included in responses:

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 999
X-RateLimit-Reset: 1642245600
```

SDKs and Libraries

Go SDK

```go
import "github.com/digital-vasic/translator-go"

client := translator.New("your-api-key")
result, err := client.Translate.Translate(
```

```
    translator.TranslateRequest{
        Text:       "Hello, world!",
        SourceLang: "en",
        TargetLang: "sr",
        Provider:   "openai",
    },
)
```

Python SDK

```python
from translator import Translator

client = Translator(api_key="your-api-key")
result = client.translate.translate(
    text="Hello, world!",
    source_lang="en",
    target_lang="sr",
    provider="openai"
)
```

JavaScript SDK

```javascript
import { Translator } from '@translator/client';

const client = new Translator({ apiKey: 'your-api-key' });
const result = await client.translate.translate({
    text: 'Hello, world!',
    sourceLang: 'en',
    targetLang: 'sr',
    provider: 'openai'
});
```

Webhooks

Configure Webhook

```bash
curl -X POST http://localhost:8080/api/v1/webhooks \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{
    "url": "https://your-app.com/webhook",
    "events": ["translation.completed", "translation.failed"],
    "secret": "your-webhook-secret"
  }'
```

Webhook Payload

```
{
  "event": "translation.completed",
```

```
  "data": {
    "request_id": "uuid-string",
    "status": "completed",
    "result": {
      "translated_text": "Здраво, свете!",
      "provider": "openai",
      "confidence": 0.95
    }
  },
  "timestamp": "2024-01-15T10:30:00Z"
}
```

OpenAPI Specification

The complete OpenAPI 3.0 specification is available at:

`https://api.translator.digital/docs/openapi.json`

You can use it with tools like:

- Swagger UI: `https://api.translator.digital/docs`
- Postman: Import OpenAPI specification
- API clients: Generate from OpenAPI spec

Testing the API

Using curl

```
# Simple translation
curl -X POST http://localhost:8080/api/v1/translate/translate \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{
    "text": "Hello, world!",
    "source_lang": "en",
    "target_lang": "sr",
    "provider": "openai"
  }'
```

Using Postman

1. Import the OpenAPI specification
2. Set your API key as an environment variable
3. Start making requests

Using the API Playground

Visit `https://api.translator.digital/playground` for an interactive API testing interface.